2.680 Labs 11 and 12 - Payload Autonomy and Herons

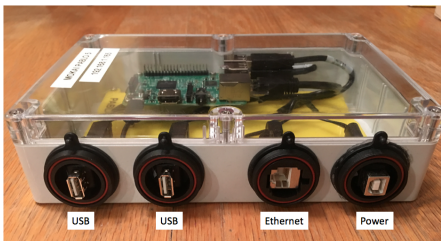
Payload autonomy and MIT Heron operation material.

Included MIT MOOS-IvP PDF sources:

- lab_class_11_pablo_intro.pdf
- lab_class_12_herons.pdf

Lab 11 - The PABLO Payload Autonomy Computer

2.680 Unmanned Marine Vehicle Autonomy, Sensing and Communications



April 2nd, 2026

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1 Overview and Objectives

This lab has the following objectives:

- Introduce the Payload Autonomy Paradigm.
- Introduce the PABLO as a hardware platform for payload autonomy.
- Access the PABLO from your laptop.
- Access the PABLO from your laptop using SSH keys.
- Download your course code (moos-ivp-extend) tree onto your PABLO.
- Identify your lab partner for upcoming in-water labs
- Coordinate shared access to you course code on the the PABLO.

By the end of this lab, you will be able to power up a PABLO computer, connected to a course laptop, log in, download and compile your course codebase. In the next lab, you will use your PABLO to run a simple mission with the Heron or BlueBoat on the Charles River.

Terms used throughout:

- **"MacOS", "OSX", "Mac OS X"**: all refer to the Apple operating system and generally Apple computers.
- **DHCP**: or Dynamic Host Configuration Protocol is a network management protocol used for automatically assigning IP addresses to devices connected to the network using a client–server architecture (Wikipedia).

Note changes for 2026: Keep these two changes in mind as you're reading the lab notes. (1) This is the first year that we include the BlueBoat as second vehicle type, whereas in past year we have used the Herons exclusively. There should be guidance for operation of both/either USV, but if you see something missing let us know. (2) On the pablos, in previous years and much of the 2.680 documentation, the username for the pablo account has been `student2680`. On the newer versions of the pablos, the user account is simply `pabuser`. If you see the former, you've probably bumped into a corner of the documentation we haven't updated yet, and you should just read `student2680` as `pabuser`.

1.1 Overview of Payload Autonomy

Consider how a robotic vessel like the Heron M300 works. Translation and rotation are effected via differential electric thrusters, which are actuated by motor controllers. Pose (e.g., heading) and position (e.g., lat/lon or x, y) are discovered by a GPS and digital compass. An on-board computer reads the sensors and commands the motors.

The sense-plan-act loop of a robot autonomy system, *could* be completed entirely within the main (front seat) computer that ships with the Heron. This arrangement, however has drawbacks. Preparing an autonomy mission with 3rd party software, like MOOS-IvP, requires access to the vehicle to load software and files, install dependencies, and build the 3rd party applications and behaviors. This effort is then duplicated on each Heron main vehicle computer. What if your dependencies are incompatible with packages installed by others? Even worse, what if the 3rd party software install breaks other aspects of the main vehicle computer environment?

Many commercially-available robotic vehicles, the M300 included, support a "front-seat, back-seat" paradigm. The front seat handles the sense and act portion of the loop, with a connected computer handling the planning and decision-making. On the Herons the only "sensing" involves the determination of vehicle position, i.e., the navigation solution. A *wire protocol* defines how the front seat and back-seat computers communicate to send sensor data from the front and to push thrust commands from the back-seat.

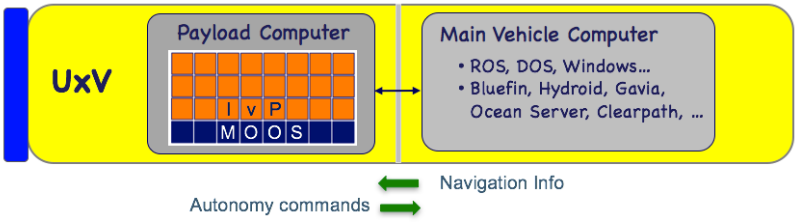


Figure 1: **The Payload Autonomy Concept.**

The `iM300` application in the `moos-ivp-heron` repository handles communicating between your mission and the M300's front seat. `DESIRED_THRUST` and `DESIRED_RUDDER` output from `pMarinePID` are converted into percent thrust to the left and right motors and sent to the front seat via the wire protocol. Pose and position messages from the front are parsed and published as the `NAV_*` variables. The vehicle plus the `iM300` app, are in effect a one-for-one replacement for `uSimMarine`.

Figure 2 offers a more detailed look of the payload/front-seat flow of information and the role if the `iM300` application.

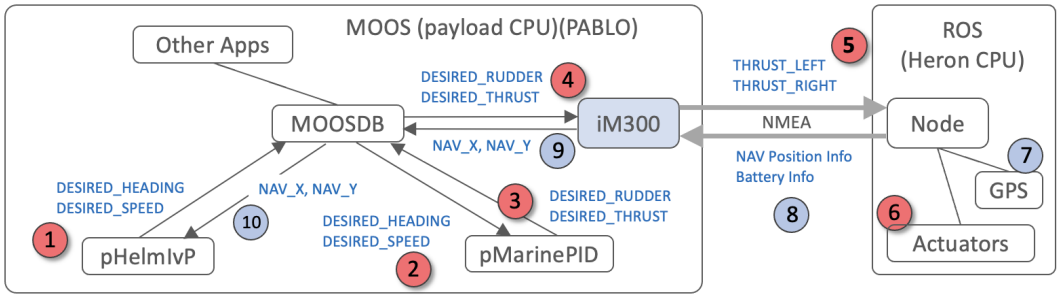


Figure 2: **The flow of information**: starting with output from the Helm, to the front-seat computer through the `iM300` application. The NMEA interface is over an Ethernet cable. This cable is a special Ethernet cable with wet-connectors on each end when used on the Herons.

1.2 PABLO Hardware Overview

The goal in this section is connect to the PABLO from your laptop. You will run an Ethernet cable from your laptop to the PABLO. And you will power the PABLO with a USB power cord. A few pieces of additional equipment are provided by the staff:

- Ethernet cable
- Ethernet to USB adaptor (USB-A and USB-C adaptors available)
- USB A to B power cable
- 5V power supply via adaptor or power strip or USB hub

There are three modes of connecting:

- Laptop connected to the PABLO with PABLO connected to the external Internet, via Internet sharing enabled on your laptop.
- Laptop connected to the PABLO with no external Internet connection on the PABLO.
- Directly connected to the PABLO a keyboard and monitor.

Our goal is the first method. The others are temporary fall-backs. You will need to have your PABLO connected to the external Internet to pull down your `moos-ivp-extend` code.

Connection steps are provided below. With MacOS there is probably less variation in the setup steps between OS versions and laptop versions. If you are working with Linux, the setup steps may have more variation. We provide the setup steps for Ubuntu 20.04, which is based on Debian. Our hope is that this matches most people, and for those using some other Linux distro, these setup steps will be similar.

1.3 PABLO Software Overview

On your PABLO computer, the Heron or BlueBoat payload computer, there will be five software trees:

1. `moos-ivp`
2. `moos-ivp-extend` (student tree)
3. `moos-ivp-2680`
4. `moos-ivp-heron` (for when using a Heron)
5. `moos-ivp-blueboat` (for when using a BlueBoat)
6. `pablo-common`

The first two trees should be familiar as you have had them already on your laptop all semester. When you receive your PABLO, it will not have the `moos-ivp-extend` tree since it is different for every person. A goal of this lab is to log onto your PABLO through your laptop with Internet sharing, so you can pull your `moos-ivp-extend` tree from `git`.

The `moos-ivp-heron` tree comes with your PABLO, compiled, and `moos-ivp-heron/bin` is already in the shell path of the PABLO user (`pabuser`). The critical app in our case is `iM300`, which implements the payload interface to the Clearpath front-seat computer. This app is discussed further in the next lab, but it essentially ingests the vehicle navigation solution and hardware status from the front-seat and publishes it in the PABLO MOOS community. And it ingests the `THRUST_LEFT` and `THRUST_RIGHT` from the back-seat PABLO computer.

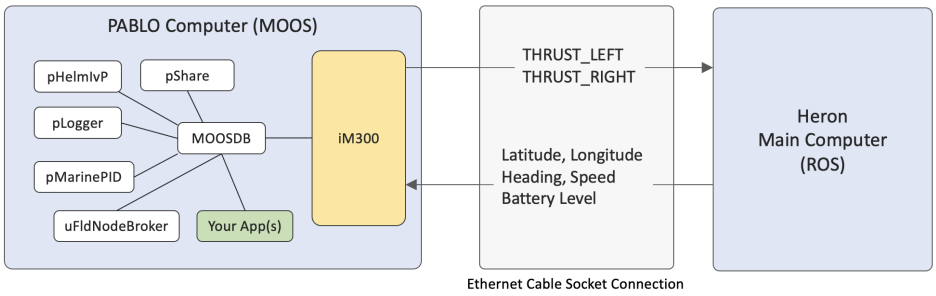


Figure 3: **The Heron Payload Autonomy Interface**: The PABLO computer box is connected by an Ethernet cable to the Heron main computer. The MOOS community running on the PABLO includes the iM300 MOOS app. This app opens a socket over the Ethernet connection to the Heron computer. It publishes the level of left/right thrust to be applied by the actuator software running on the main computer. It consumes navigation and robot status information from the Heron and publishes this information locally in the MOOSDB on the PABLO, for consumption by the helm and other apps.

The fourth tree on your PABLO, `pablo-common`, is designed to contain as much as the PABLO and `pabuser` configuration content as possible. For example, imagine creating the `pabuser` user account, with a nice `.bashrc` file, and then replicating the SD card to 50 PABLOs, all boxed up and delivered to users. Then imagine that something was wrong or missing in the `.bashrc` file. At this point our options would be to either (a) log into all 50 boxes and edit the file, or (b) fix it on one box and clone 50 new SD cards and swap out all cards. Neither option is sane.

Instead, we put the bulk of the `.bashrc` file in another file in `pablo-common`. The *actual* `.bashrc` file on the PABLO is quite thin and simply reads further `bash` configuration content from another file in `pablo-common`. In this way, a change to the `bash` environment for all PABLOs can be made in the `pablo-common` tree, and pushed into version control. The PABLO user can simply pull the change. In our case, the PABLO is configured to automatically pull `pablo-common` changes when it boots and finds the Internet.

1.4 Working with the PABLOS and the Heron Robots

When we begin working with the Heron robots on the water, your PABLO will be running the vehicle. The Ethernet is connected between the PABLO and the Heron front-seat computer. And the PABLO is powered through the USB port from a small battery in a similarly water-tight box, as shown on the right in Figure 4. The gap between the PABLO and the Heron is open to the elements and that's why all the connectors on the PABLO are Bulgin IP67 rated water-tight connectors.

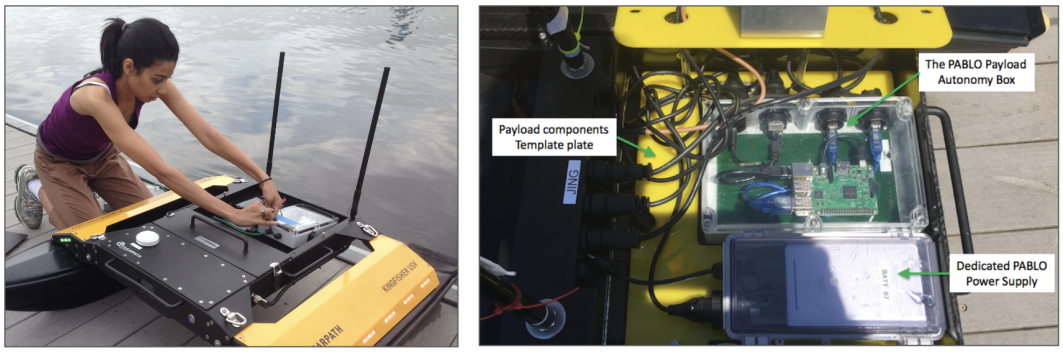


Figure 4: **The PABLO with the Heron USV**: (left) In 2015, an earlier version of the PABLO was used in the Heron. (right) The current version of the PABLO has a slightly different connector layout and the payload compartment is structured to fit the PABLO box in a pre-defined location. The PABLO battery is also now in its own battery box with wet connectors.

The two USB A ports on the PABLO will need to be capped when you work with the vehicle on the water.

With the PABLO, the idea is that you (a) prepare your code in simulation on your laptop, and push it to version control, (b) connect your PABLO to the laptop and pull your code onto the PABLO and compile. And finally, when you're ready to test on the water, (c) find a Heron, connect your PABLO and launch your mission. So far we have been solely working in mode (a), in simulation on your laptop. Connecting your PABLO and pulling and preparing your code is a key step, and is the focus of today's lab.

When you do eventually connect your PABLO to the Heron, it will still be possible to log onto your PABLO and pull code updates and re-compile. However, robot time is a precious commodity, especially dock-side on a day when testing is supported by colleagues. You will want to have as much prepared on your PABLO as possible before you arrive at the dock.

2 Connecting to the PABLO through a Mac Computer

For the below instructions, things are slightly different in terms of the layout in `System Settings` menus between MacOS Sonoma or MacOS Sequioa (newest) releases. Where possible we try to show both views, but they really are pretty similar enough that you should be able to find your way around, or with TA help. You can always find out which release you have by selecting the Apple pull-down menu that persists in the upper left-hand corner of your window, and select `About This Mac`.

For MacOS users, your Mac will effectively act as a router, providing the PABLO with an IP address, connectivity to the computer, and a route to reach the Internet. MacOS has a feature called *Internet Sharing*. This option allows the Mac to get a connection to the Internet on one interface (e.g., wireless) then allow another device to connect to the Mac over a different interface (e.g., Ethernet). Effectively, the PABLO will have access to the Internet and to the Mac by plugging into the Mac's Ethernet port.

When Internet Sharing is turned on, the Mac creates a new subnet and by default assigns itself the shared connection the IP address `192.168.2.1`. The connected PABLO device will be assigned `192.168.2.2`. However, like any router you are not always guaranteed to receive the `.2` suffix on the IP address. We describe below how to detect the IP address that was actually assigned to your PABLO.

2.1 Enabling Internet Sharing on the Mac

Below are the steps for enabling Internet Sharing on a Mac:

Step 1: Open Systems Settings: Click the apple menu (top-left of the menu bar), and select *System Settings*.

Step 2: Open the *Sharing* sub-page: Click on `System Settings` then `General`, then `Sharing`. It should like Figure 5 below.

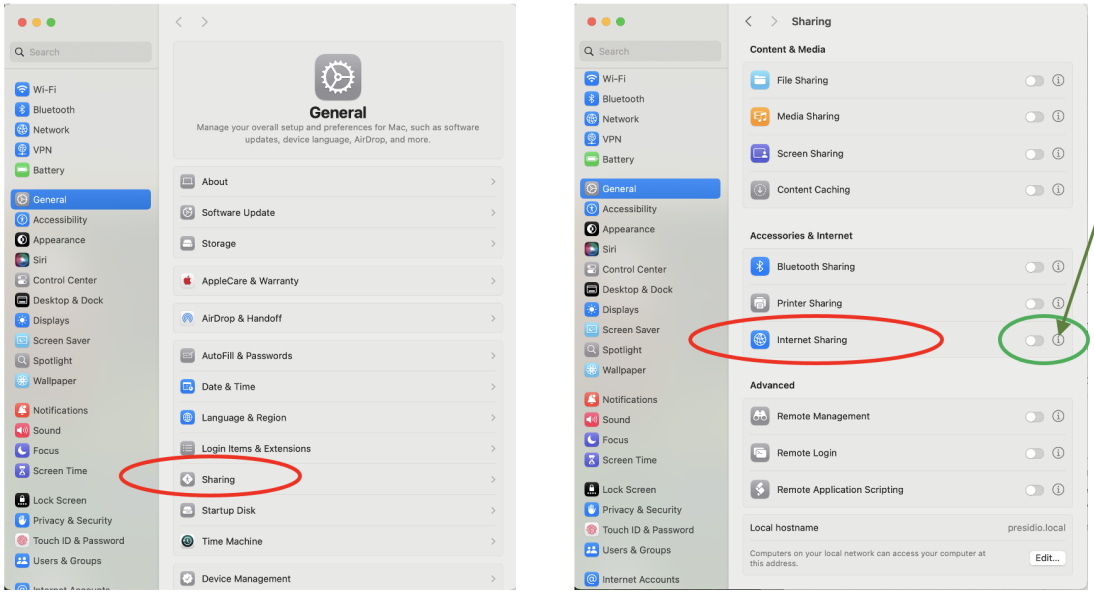


Figure 5: On MacOS Sequoia release, System Settings will look like this.

Step 3: Configure *Sharing*: Disable Internet Sharing if it is currently enabled. See the toggle button at the end of the arrow in Figure 5. When it is disabled, you can make selections on its configuration by clicking on the (i) button next to the toggle button as shown in the same figure. For the *Share your connection from:* option, select *Wi-Fi*. For the *To computers using:* option, check the box or boxes related to Ethernet. These may have names like `USB10/100/1000LAN` as shown in Figure 6.

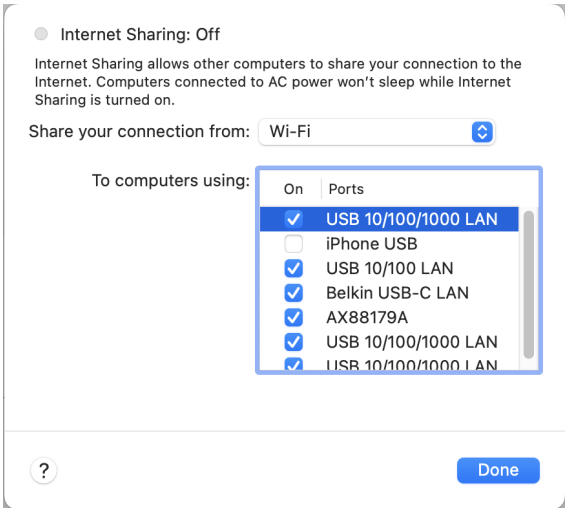


Figure 6: The settings for Internet Sharing may be selected as shown on the right. The Mac will connect to the Internet through Wi-Fi, and share with computers connected to one of the Ethernet connector options.

Step 4: Enable *Sharing*: Once you have the settings the way you like, go ahead and check the box next to Internet Sharing, as shown at the end of the arrows in Figure 5.

You are now ready to connect your PABLO. (1) Connect the Ethernet cable between the PABLO and your Mac. (2) Then power on the PABLO by plugging the USB cable into a power source and then into your PABLO.

Step 5: Disable *Sharing*: When you're done working with the PABLO, de-select the *Internet Sharing* option.

2.2 Detecting the PABLO IP Address in MacOS

With Internet Sharing on the Mac turned on, when you connect the PABLO to the MAC it will be assigned an IP address. The assigned IP addresses are stored in a system file. To determine which IP address was assigned to your PABLO, you can just display the contents using the `more`, or `cat` command:

```
$ cat /var/db/dhcpd_leases
{
    name=PABLO
    ip_address=192.168.2.2              <- This is your PABLO's IP address
    hw_address=1,dc:a6:32:4b:88:c0
    identifier=1,dc:a6:32:4b:88:c0
    lease=0x62535727
}
```

A couple things to note about this file.

- It persists even after you disconnect your PABLO. If you reconnect the same PABLO, or power off/on the PABLO while retaining the Ethernet connections, your Mac can tell it's the same PABLO since it also detects the unique hardware address in the `hw_address` field above.
- If you connect a new PABLO, your Mac will assign the next available IP address, likely `192.168.2.3` in this case. This file will now contain two blocks like the example above. The newest block is usually at the top.
- You can safely remove this file at any time.

  ```
  $ cd /var/db
  $ sudo rm dhcpd_leases
  ```

  The next time you re-connect a PABLO to your Mac, the file will be written to again with the latest assigned DHCP lease, starting again with `192.168.2.2`

You are now ready to proceed to Section 4.

3 Connecting to the PABLO through a Linux Computer

These instructions are from the perspective of Ubuntu 20.04. The steps may vary for other Linux distros.

3.1 Enabling Internet Sharing in Ubuntu 20.04

Below are the steps for enabling Internet Sharing in Ubuntu 20.04.

Step 1: Open System Settings Main Menu: Select the menu item on the left labeled `Network` as shown in Figure 7 below. NOTE: On some computers, if the Ethernet cable is not yet physically connected, the `Wired` section may simply just not shown.

After you connect your Ethernet cable from your Linux machine to the powered-on PABLO, the *Wired* menu will be enabled and will look similar to Figure 7 on the right.

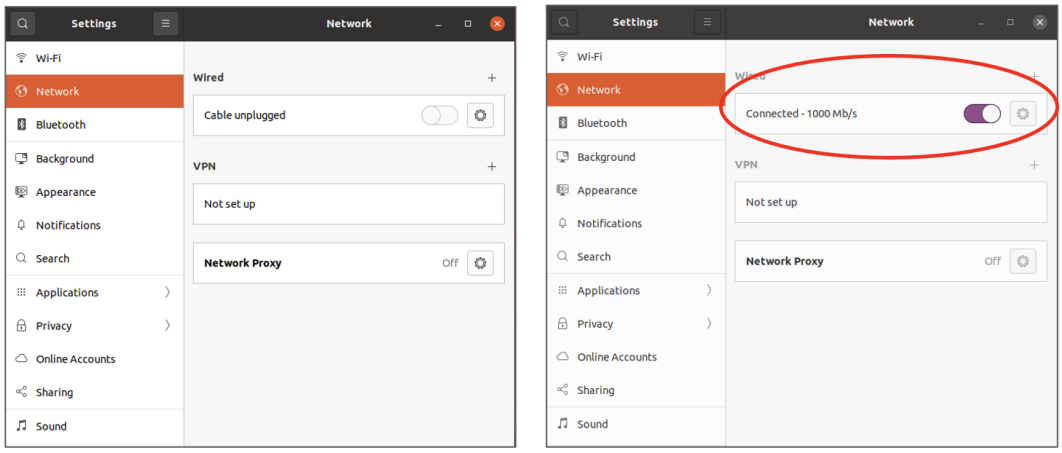


Figure 7: **The Ubuntu Settings Network Menu Option**: The Wired option in the Network menu will indicate an unplugged cable (left). Note: The Wired section may also be just not shown on newer versions of Ubuntu. Either way, plug in the cable. When the Ethernet cable is inserted and connected to another device, the Wired menu item changes automatically to be enabled (right).

Step 2: Click on the Gear Icon in the Wired Menu Item: By clicking on the gear icon, shown on the left in Figure 8, the dialog window for configuring the *Wired* connection will open, shown on the right in Figure 8.

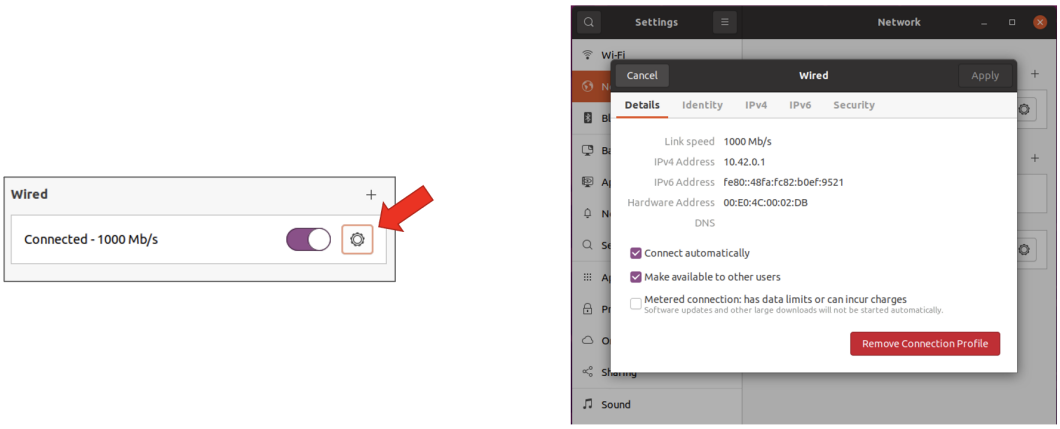


Figure 8: **Editing the Settings for the Wired Connection**: Click on the gear icon in the Wired Menu item (left). A dialog window will open to configure the Wired parameters (right).

Step 3: Choose IPv4 Settings: Select the IPv4 tab and use the settings *Shared to other computers*

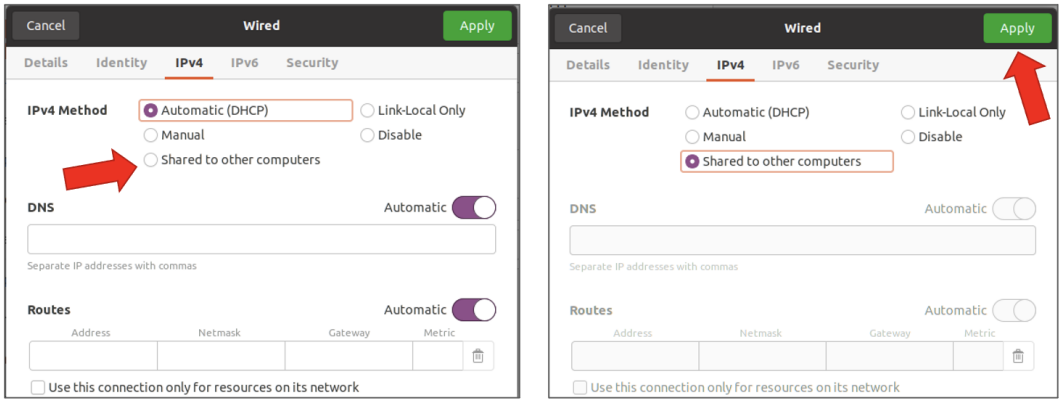


Figure 9: **IpV4 Settings**: Initially the Sharing option is disabled (left). After selecting it and hitting the Apply button, it will be enabled.

Step 4: Connect the Ethernet Cable: If you haven't done so already, connect the PABLO and laptop together over Ethernet, and power on the PABLO. When they are not connected the *Wired* dialog box, with the *Details* tab, looks like the left in Figure 10. When it *is* connected, it should look like the right in Figure 10. Notice on the right, the Ethernet IP address of the Linux machine is shown. In this case `10.42.0.1`.

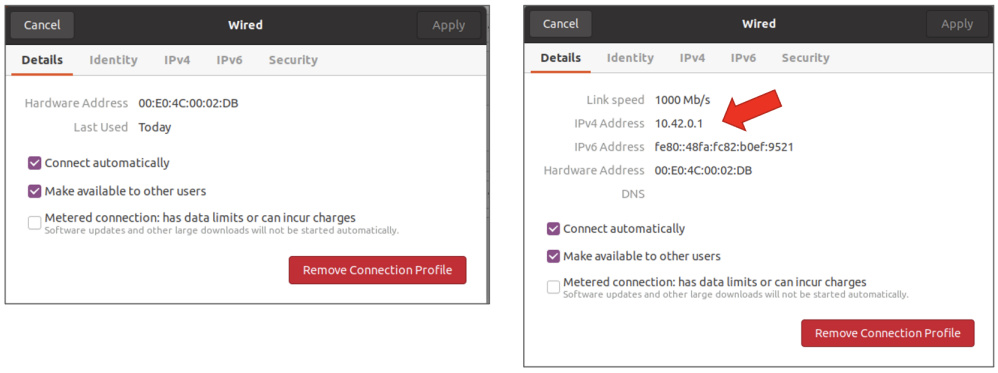


Figure 10: **IPv4 Address**: When the Ethernet is not connected, the *Details* dialog box looks like the left. Once the Ethernet is connected to a powered-on PABLO, it looks like that shown on the right, displaying the Ethernet IP address.

The next step, discussed next, is to determine the IP address of the PABLO, so we can connect to it and log in.

3.2 Detecting the PABLO IP Address in Linux

There are a couple options (and probably more that I'm not aware of), for determining the IP address that was assigned to the PABLO. We describe both. The first is perhaps a bit simpler but may not work on systems other than the Ubuntu 20.04 that I have tested on. But it's so simple, so why not.

Method 1: Determine the PABLO IP: On the command line, try the below:

```
$ cat /var/log/syslog | grep DHCPACK
Apr 11 16:28:40 pa dnsmasq-dhcp[16042]: DHCPACK(en..db) 10.42.0.236 dc:a6:32:4b:88:00 PABLO
Apr 11 16:58:45 pa dnsmasq-dhcp[16042]: DHCPACK(en..db) 10.42.0.236 dc:a6:32:4b:88:00 PABLO
Apr 11 17:05:35 pa dnsmasq-dhcp[16332]: DHCPACK(en..db) 10.42.0.236 dc:a6:32:4b:88:00 PABLO
```

You should be able to pick out the IP address `10.42.0.236` from the above, or you can use more command line magic:

```
$ cat /var/log/syslog | grep DHCPACK | tail -1 | cut -d ' ' -f7
10.42.0.236
```

Hint: If the above step works for you, consider adding an alias in your `.bashrc` to make it easy in the future.

As mentioned, the above may or may not work on your Linux distro. With Ubuntu 20.04, it does seem that new DCHP leases are logged in the system file `/var/log/syslog` in this manner. If this doesn't work, try the next method below.

Method 2: Determine the PABLO IP: In this method, we use the nmap utility. If this is not installed on your Linux machine, install it now:

```
$ sudo apt install nmap
```

On the command line run nmap as follows:

```
$ nmap -sP 10.42.0.1/24
Starting Nmap 7.80 ( https://nmap.org ) at 2024-04-03 21:50 EDT
Nmap scan report for rehoboth (10.42.0.1)     <--- YOUR Linux Machine
Host is up (0.00020s latency).
Nmap scan report for 10.42.0.236              <--- The PABLO
Host is up (0.00061s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 3.03 seconds
```

The nmap command will respond with a list of hosts that are up. One will be the laptop you are using while the other should be the PABLO's IP. *In the following instructions do not use 10.42.0.236 for the PABLO's IP but the one that* nmap *just found!*

4 Verifying PABLO Connection and Login

Now that your PABLO is connected to your laptop, the next step is to verify that you can log in. You should at this point be armed with your PABLO's IP address, e.g., something like `192.168.2.2` on the the Mac, or something like `10.42.0.236` on Linux.

4.1 Using ping Command Line Utility

The ping command is a command-line (OS X and GNU/Linux) tool for testing whether another computer can be reached across an IP network. After connecting the PABLO, it is useful to use ping to verify that PABLO can be reached from your laptop. In a terminal window on your laptop:

```
$ ping 192.168.2.2
PING 192.168.1.2 (192.168.1.2): 56 data bytes
64 bytes from 192.168.1.2: icmp_seq=0 ttl=64 time=9.203 ms
64 bytes from 192.168.1.2: icmp_seq=1 ttl=64 time=12.605 ms
64 bytes from 192.168.1.2: icmp_seq=2 ttl=64 time=7.238 ms
^C
--- 192.168.1.2 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 7.238/9.682/12.605/2.715 ms
```

The ping command can be stopped by pressing `control-C`. In the example above, ping reached the PABLO and reported the round-trip communications time. When the connection fails, output similar to the following is displayed:

```
 $ ping 192.168.2.2
 PING 192.168.2.2 (192.168.2.2): 56 data bytes
 Request timeout for icmp_seq 0
 Request timeout for icmp_seq 1
 Request timeout for icmp_seq 2
 ^C
 --- 192.168.2.2 ping statistics ---
4 packets transmitted, 0 packets received, 100.0% packet loss
```

4.2 Assignment 1 (Self Check off) SSH onto the PABLO

The `ssh` tool is a command-line (OS X and GNU/Linux) tool for opening up a terminal on a remote device using a secure shell connection. In other words, `ssh` connects the keyboard and terminal window directly to another computer that is reachable across the network (i.e, `ping` displays an affirmative response). Inputs to `ssh` include the IP address of the remote system and the user name. In a terminal window on your laptop, enter the following to connect via `ssh`:

```
 $ ssh -l pabuser 192.168.2.2
```

The username is `pabuser` and the password will be provided by the lab instructor. Once connected, the terminal will behave as if the keyboard and screen (at least the terminal window on the screen) were connected directly to the remote system.

Final part of this step: Once you are logged into the PABLO, verify that you have a connection to the external Internet through your laptop. We can do this by pinging the course server

```
PABLO $ ping oceanai.mit.edu
PING marine-robotics.mit.edu (18.18.38.22): 56 data bytes
64 bytes from 18.18.38.22: icmp_seq=0 ttl=55 time=18.218 ms
64 bytes from 18.18.38.22: icmp_seq=1 ttl=55 time=21.325 ms
64 bytes from 18.18.38.22: icmp_seq=2 ttl=55 time=21.999 ms
```

If you do not see a successful ping as above, double-check that you have Wi-Fi enabled on your (laptop) machine. If you do see a successful ping as above, you are ready for the next step.

Note: Hereafter we use the following shell prompt to indicate you are working on your laptop:

```
 $
```

And the following shell prompt to indicate you are working on your PABLO:

```
PABLO $
```

5 Accessing your Course Code on the PABLO

In this section the goal is to get your `moos-ivp-extend` code onto the PABLO. The `git` command should be installed on your PABLO. If for some reason it is not, let us know. It can installed on your PABLO with:

```
PABLO $ sudo apt-get install git
```

5.1 Assignment 2 (Check off) Loading, Building Your Tree on the PABLO

In this step the goal is to (a) load your personal repository onto PABLO unit, (b) build the tree, (c) add your `bin/` directory to the shell path, (d) update your `IVP_BEHAVIOR_DIRS` environment variable, and finally (f) demonstrate that your binaries are runnable from the command line.

Step 1: Log back on to the PABLO If you're not still logged in, log back in:

```
$ ssh -l pabuser 192.168.2.2
pabuser@192.168.2.2's password:
Last login: Sat Mar 23 17:47:24 2023 from 192.168.2.1
PABLO $
```

Step 2: Check out your tree: By default, logging into the PABLO puts you in the `pabuser` home directory. Check out your git repository into that home directory.

```
PABLO $ git clone git@github.com:janesmith/moos-ivp-janesmith.git
```

Don't forget that on the PABLO you are the user `pabuser`. On your laptops you may have the same user name as your git account, and you may need to keep this in mind when pulling down your code. You may need to add the user `pabuser` to your version control access list.

Step 3: Build your tree: The `moos-ivp` and other trees should already be loaded on the machine. To build your tree:

```
PABLO $ cd moos-ivp-janesmith
PABLO $ ./build.sh
```

Step 4: Augment your Shell Path Configure the system path to know where your MOOS applications are located. Using the editor of your choice, on the PABLO unit edit the file `/home/pabuser/.bashrc`. At the end of the file, add:

```
export PATH=$PATH:/home/pabuser/moos-ivp-janesmith/bin
```

Source the `.bashrc` file (or log out and back in) to apply changes to the `PATH`.

5.2 Assignment 3 (Check off) Decide on a Lab Partner

For the remainder of the labs, you will be working with a lab partner. Decide who this is, and let the TAs know.

5.3 Assignment 4 (Self Check off) Access your Lab Partner's Code

As you prepare for the in-water labs, you will want to test in simulation. You will need to work collaboratively on a shared set of code. Work out with your lab partner how you want to do this. It's OK to just share one person's code base. Even better if you each have access to each other's codebase and you have both codebase on each of your PABLOs.

5.4 How to Best Shut Down the Pablo Box

When you are done, or anytime you need to shut down the Pablo Box, run the following command before disconnecting power:

```
PABLO $ sudo halt
```

This ensures the computer will nicely shut down before power is removed.

5.5 Assignment 5 (Bonus to Yourself) Streamline Your Workflow

Normally when you log onto a PABLO installed on a Heron robot, it is a two step process:

```
  $ ssh -l pabuser 192.168.14.100    (For vehicle abe)
```

You have to (a) remember the IP address for vehicle abe, and (b) enter the password for the user `pabuser`. That's a lot of typing. It can be done, but it can be simplified to:

```
$ ssha
```

With the steps below, you can just type these four characters, with no need to enter a password. Nothing to remember, except `ssha` is for `"abe"` and `sshb` is for `"ben"` and so on.

5.5.1 Install SSH Keys

When labs move to the water, you will want to `ssh` onto your PABLO many times. Typing in the password will be a nuisance. Create an ssh key pair and install the public half in the `/.ssh/` directory on your PABLO.

https://oceanai.mit.edu/ivpman/help/sshkeys

5.5.2 Add Heron Aliases to your Laptop Bash Environment

When operating on at the Pavilion, with your PABLO installed into one of the Herons, the PABLO will be on the local WiFi network with an IP address determined by which Heron it is on. The names of the MIT Herons are:

- abe
- ben
- cal
- deb
- eve
- fin
- max
- ned
- oak
- pip

If you add the following aliases *on your local laptop*, it will be much easier to log into your PABLO. Typing `ssha` for example takes you right to the PABLO on the vehicle `abe`. AND, if you have your ssh keys installed, then logging in to your PABLO is only 4 key-strokes away! It may seem minor, but when operating robots in the field, experienced users take every opportunity to simplify routine steps.

```
alias  ssha='ssh -l pabuser 192.168.14.100'
alias  sshb='ssh -l pabuser 192.168.15.100'
alias  sshc='ssh -l pabuser 192.168.16.100'
alias  sshd='ssh -l pabuser 192.168.17.100'
alias  sshe='ssh -l pabuser 192.168.18.100'
alias  sshf='ssh -l pabuser 192.168.19.100'
alias  sshm='ssh -l pabuser 192.168.20.100'
alias  sshn='ssh -l pabuser 192.168.21.100'
alias  ssho='ssh -l pabuser 192.168.22.100'
alias  sshp='ssh -l pabuser 192.168.23.100'

alias pinga='ping 192.168.14.100'
alias pingb='ping 192.168.15.100'
alias pingc='ping 192.168.16.100'
alias pingd='ping 192.168.17.100'
alias pinge='ping 192.168.18.100'
alias pingf='ping 192.168.19.100'
alias pingm='ping 192.168.20.100'
alias pingn='ping 192.168.21.100'
alias pingo='ping 192.168.22.100'
alias pingp='ping 192.168.23.100'
```

6 Troubleshooting Notes

6.1 Mechanical Troubleshooting

If things aren't connecting, consider the set of things that must work, and methodically swap out with spare components. Besides the PABLO itself:

- Swap out the Ethernet cable
- Swap out the Ethernet to USB adaptor (if you're using one)
- Swap out the PABLO power cable. You should be able to look inside the PABLO and see the red and green lights come on. If this is happening, then you're USB power cable is 99 percent probably OK.
- If you do not see the LEDs come on inside the PABLO, you may have a bad USB power cable. If you are using a USB power hub, check that the hub itself is powered on. The ANKER 10-port hubs have a separate power switch on back, and there is a blue LED on the front of the ANKER hub when powered on.
- Of course you can also try another PABLO box.
- If it turns out the original PABLO box you were given was not working, you can un-screw the top and check that all the internal connectors are snug.

6.2 On the MAC - No entry in the dhcpd_leases file

If you're not seeing evidence of your PABLO in the form of an entry in the `/var/db/dhcpd_leases` file:

- Check your spelling of this file
- Bring down the Internet Sharing in the Systems Settings window. Click on the Info button to the far right of the Internet Sharing menu item (the little circle with the 'i' in it, as shown the small green circle on the right in Figure 5). Check all boxes for sharing that look like they have anything to do with Ethernet, e.g., `Belkin USB-C LAN`, or `USB 10/100/1000 LAN`. Then un-power your PABLO and re-insert the USB power cable, ensuring your Ethernet cable is snug on both ends.

6.3 On the MAC - Multiple entries in the dhcpd_leases file

If you see multiple entries in this file, your PABLO, if it is successfully connected, should be the highest number, e.g., `192.168.2.3` and not `192.168.2.2`.

This file can be removed if you feel it's warranted. It will be auto-generated the next/first time a device is connected to your Mac with Internet sharing enabled.

6.4 Connecting to the PABLO - Fall-back Methods

When the PABLO boots, it will attempt to get an IP address for its Ethernet port from a DHCP server. When the PABLO boots, if it cannot obtain an IP address via DHCP, the Ethernet (`eth0`) interface will fall back to having the IP address of `192.168.2.24`. So a fall-back option for connecting

with your laptop, or any machine over a direct Ethernet connection, is to set your machine's Ethernet IP address manually to say `192.168.2.6`, and just connect with:

```
$ ssh -l pabuser 192.168.2.24
```

If the above doesn't work, try `ping 192.168.2.24`. If `ping` works then re-check your typing in the `ssh` command. If this option or any of the below methods do not work, your ultimate fallback is to connect with a monitor and keyboard and check out what is going on by running `ifconfig` once you've logged on to the PABLO.

6.5 Check the PABLO Power Source

When the PABLO boots, initially you should see a small red LED turn on if you look down though the lid. A green light should also blink periodically as it boots and begins to handle network traffic.

Consider what you are using for a power source. Even though power-over-USB should allow you to connect your USB power cable to your laptop, the Pi may experience a brown-out at some point.

Lab 12 - Payload Autonomy on an M300 Heron USV

2.680 Unmanned Marine Vehicle Autonomy, Sensing and Communications

April 9th, 2026

Michael Benjamin, mikerb@mit.edu
Tyler Paine, tpaine@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139

1 Overview and Objectives

The objective of this lab is to prepare students for deploying vehicles with a PABLO unit at the MIT Sailing Pavilion. This lab is a companion to the PABLO introduction lab, which describes the interface to the PABLO unit and how missions are structured for on-vehicle deployment. The sections below introduce the deployment environment, describe the hardware, and explains how to transition from simulation to real vehicles. By the end of this lab, the student will be able to launch a mission on a PABLO unit that drives an M300 vehicle across the river while being monitored from a shoreside laptop. Steps to be completed in this lab include:

- Introducing the Marine Autonomy Lab at the MIT Sailing Pavilion
- Introducing the Clearpath Heron M300 vehicles
- Understanding the computer network environment at the Pavilion
- Editing the alpha_heron mission to run on actual hardware

2 The Marine Autonomy Lab at the MIT Sailing Pavilion

The Marine Autonomy Laboratory at the MIT Sailing Pavilion provides a convenient waterside location to test marine autonomy software on real-world vehicles. To reach the lab, enter the main entrance of the Pavilion, descend the stairs onto the dock, and turn left. Doors are kept locked when not in use. Coordinate with lab staff to gain access.

While deploying vehicles at the Pavilion, please be courteous to other users of the facility and boaters on the river. Mornings generally have less water traffic and fewer people at the dock. Sailing teams from MIT and other organizations are active in the afternoons and early evenings. Rowing crews are active in the morning and a lookout needs to watch for their boats. *Sailing classes, regattas, and Pavilion activities always have priority over deploying robotic boats.* The boats are deployed in all weather except for extreme cold and during thunderstorms. There is never a guarantee of water time.

Never Work Alone: If you arrive early or find yourself alone in the lab, please be conservative in your activities. It's ok to hang out and work on your laptop alone, but NOT OK to be accessing the water or putting equipment in and out of the water alone.

3 Robotic Vehicles at the Pavilion

The current fleet stored at the Pavilion includes nine robotic vehicles, Clearpath Heron M300 vessels, that make take on the names of either abe, ben, cal, deb, eve, fin, max, ned, oak

3.1 Introduction to the Heron M300 Vehicle

Figure 1: The Clearpath Robotics Heron M300 on the river.

The Clearpath Heron M300 is a collapsible catamaran-style vessel that can be deployed by a single person, although it is easier to handle with two people. Jet-drive motors are installed at the stern of each pontoon that allow efficient forward thrust and very modest reverse capability. The main body of the vehicle has three sections:

Section	**Description**
Computing	The forward section contains the front seat embedded computer with a 2.2GHz Intel ATOM processor and support equipment for motor control, communications, and sensing.
Battery	A single battery package is installed above the main electronics bay in the front section. It is used to power propulsion and for computing.
Payload	The large aft-ward tray is the payload bay where the back seat computer (the PABLO unit) is housed. This section has removable top and bottom covers. The bay includes three waterproof ports: two USB-A connectors and an Ethernet receptacle.

Table 1: Sections of the M300 vessel

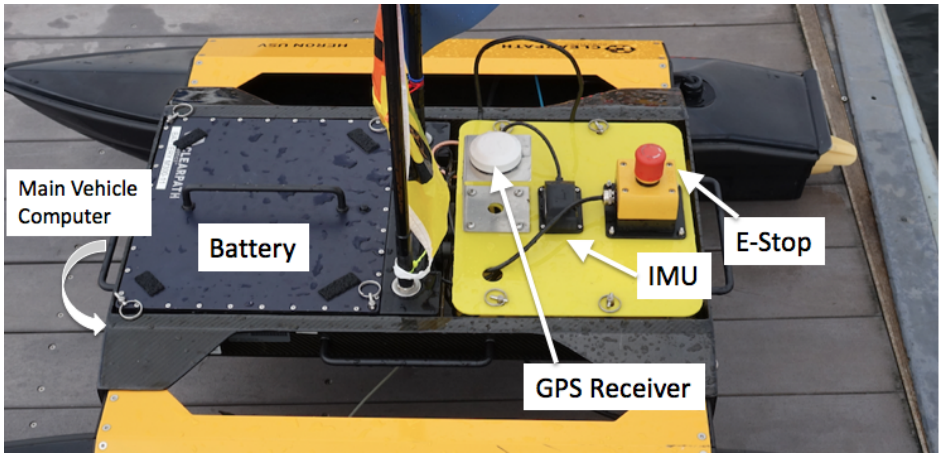


Figure 2: Some main components of the M300 USV with payload, sensor modifications for the MIT variant. The E-Stop equipment is unique to the MIT variant. The GPS and IMU sensors ship with the vehicle, but their locations and mounting have been modified in the MIT variant for better performance and access.

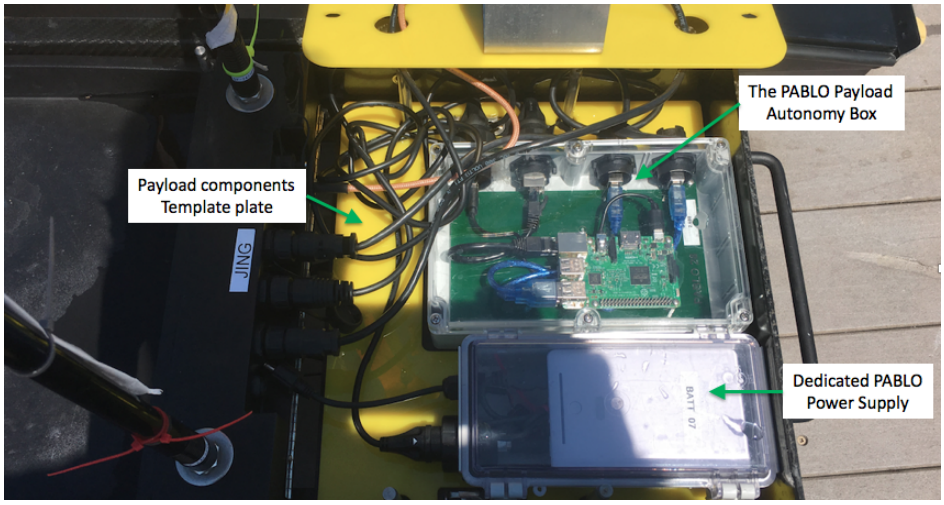


Figure 3: In the MIT variant, the payload compartment contains both the payload autonomy box (PABLO) and a dedicated power supply for the PABLO (the smaller box in the photo). Both components sit in a custom template cut with holes to fit the PABLO and battery box. The latter helps to ensure a position for smooth cable arrangements.

Navigation Sensors

The GPS antenna is a white puck mounted on the lid of the payload bay cover as in Figure 2. An inertial motion unit with integrated digital compass is also mounted on the payload cover lid. Both sensors connect to and are processed by software on the frontseat computer. Output from these sensors is published by the frontseat, as defined by the Clearpath Wire Protocol, shared to the

PABLO computer running the payload autonomy (MOOS-IvP) software.

Vehicle Batteries and Charging

The lab has a number of battery packs available for use. Several of the batteries are suitable for on-dock preparations but do not include enough capacity for sustained deployment. Batteries are stored and charged on the metal rack at the rear of the lab. One battery package is charged by two charger units. To charge a battery, set it on the side so that the connector is on the top. Plug in the charger with the red plugs matching. If necessary, orient the chargers so that the LED indicators are facing outward. Red or amber light indicates charging; completed charging shows a green light.

Installing Batteries

Always pull the M300 out of the water and well onto the dock to change the battery. **Do not change the battery while the vessel is in the water**. Change the batteries in a place where there is no chance of dropping the battery into the river.

The M300 battery is secured by four pins, each with a circular key-ring at the end, as shown in Figure 2. To remove a battery, undo the four pins and pull upward on the handle. If it is stuck, give a light yank or seek assistance from the lab instructor. To install a battery, first verify that the battery terminals align with the power port on the vehicle. Insert the battery straight down and push it in until it is almost flush with the vehicle. After the battery is firmly in place, install all four pins on each of the four posts. **Do not power on the vehicle unless the battery is secured.**

3.1.1 Storing and Moving

The M300 units are usually stored on carts or on a shelf inside the lab, although it is acceptable to leave units on the floor at the rear of the lab. With only one handler it is advisable to remove the battery before lifting or moving the vehicle. With two people the battery's additional weight is typically manageable. The vehicle carts are provided for moving the M300s around the dock and lab space. Take care to secure the cart if is used on the docks, to make sure they don't roll away. They will sink if rolled into the river (in theory - don't be the one that proves this theory).

3.1.2 Deploying and Retrieving

Always have the correct remote control handy when the M300 is in the water near the dock. To deploy, take the vehicle off the cart and set it on the dock with the thrusters pointing into the river. Verify that the remote can control the thrusters. Gently push the vehicle into the water and use the remote to transit away from the dock. It is inadvisable to leave the vehicle drifting near or tied to the dock. Wake from passing motorboats can pummel the vehicle into the floating platform.

To retrieve an M300 vessel, use the remote control to bring it back to the dock head first. Lift and pull it onto the dock by the front handle. Using the side handles, one or two people can lift it onto the cart.

3.1.3 Manual Remote Control

Remote control (RC) operation is suitable for transiting from or returning to the dock or as an emergency override. The flat slider switch adjacent to the LCD display turns the unit on and off. When the RC is powered on, the switch on the upper-left (SW2) of the face selects when the vehicle's computer (switch down) or the remote (switch up) has control. When enabled, the RC will override the on-board computer. RC range is approximately 100 feet. When not in use, the remote control should be turned off and connected to charge on its hook at the rear of the lab.

3.2 Boxes vs. Bodies

The main computer on the Heron vehicles comes in an electronics box that can be swapped between vehicles. This is a wonderful design feature that allows us to keep more vehicles in a ready state. Each electronics box costs about 1/6 the cost of the overall Heron. So we tend to have more electronics boxes than bodies. It also raises the existential question of "Where does the identity of the vehicle reside? In the body or the box?"

Because a lot of the vehicle identity is tied to the network IP assignments, we tend to associate the identity of the vehicle with the (much smaller) electronics box. Each electronics box is labeled. Just keep this in mind, since the vehicle frame and pontoons sometimes may have a conflicting label or flag mounted on an antenna.

You will also want to be aware of the vehicle identity when it comes to grabbing a remote control hand unit. These are labeled with the vehicle name and correspond to the electronics box.

3.3 Assignment 1 (check off): Manual Control of an M300

Before connecting a PABLO unit to the vehicle, the lab instructor will walk you through manually deploying and controlling the M300 vehicle. Go through the following checklist:

In the lab...
[ ] Identify a vehicle to use with one of the TAs
[ ] Install battery in vehicle and secure all four latch pins
[ ] *Verify that unused payload bay connectors are securely tightened*
[ ] Securely install the payload cover
On the dock...
[ ] With a partner, place vehicle gently on dock near the water
[ ] Power on the vehicle
[ ] Verify that the remote control actuates thrusters
Deploying...
[ ] Push vehicle into water stern first, then try to hand-rotate pointing it away
[ ] Drive vehicle away from dock
[ ] Attempt straight line for 10 meters
[ ] Attempt tightest turn to right, left, rinse and repeat.
Retrieving...
[ ] Carefully return vehicle to dock (do not hit the dock)
[ ] Lift vehicle onto dock and leave it powered on

4 Connecting to WiFi at the Pavilion

There are several WiFi networks available at the Pavilion. The `kayak-local-5GHz` and `kayak-local-2GHz` networks are used for this class. The access-point WiFi router is located inside the lab and provides wireless coverage for laptops in the lab, classroom, and a portion of the outside dock. Connection to the 5GHz network is preferred.

WiFi coverage to vehicles on the river is possible with a dedicated wireless bridge. The sector repeater and antenna must be deployed and powered so that vehicles can communicate on the network. Remember to unplug the antenna after shutting down the vehicles.

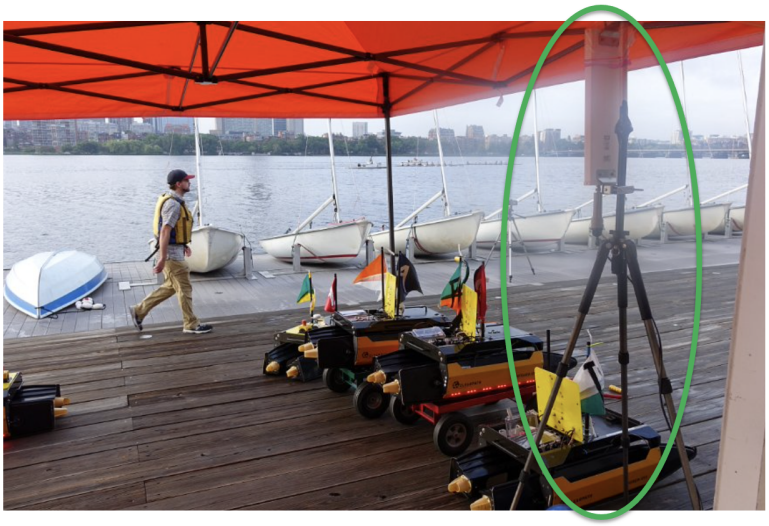


Figure 4: The pavlab antenna is mounted on a tripod as shown here. Normally the antenna is stored just inside the doors of the lab. Sometimes the tripod will be mounted up on the sailing pavilion roof.

Computers and vehicles connected to the lab's WiFi operate on a dedicated subnet. Devices can all see each other and originate network traffic to the Internet but are insulated from outside requests. Cabled or WiFi connections to the router receive a local IP address like `192.168.1.xxx`, where `xxx` is a number unique to each device. Vehicles and lab-related computers have static IP addresses that will remain the same.

4.1 Your Laptop IP Address

On your laptop, the IP address may change between visits to the lab. The number should remain constant for the entire day. You probably won't need to know which IP address your laptop has obtained. You will be more concerned about reaching the robot computer from your laptop. If you're curious you can check the Network section of your Systems settings. Or to find the address assigned to your computer, use the `ifconfig` command in combination with `grep`:

```
$ ifconfig | grep 192.168
inet 192.168.1.208 netmask 0xffffff00 broadcast 192.168.1.255
```

In this example, the assigned address is `192.168.1.208`.

4.2 Heron IP Addresses

Each Heron has three IP addresses. (a) The first IP address allows the user to log into the Heron front-seat computer. This IP address is a static IP address configured in the router on the wall in the lab. (b) the second IP address is the IP address of the PABLO if you were logged in to the front-seat computer. We will rarely use this address. (c) The third IP address allows the user to directly log into the PABLO without needing to first log in to the front-seat computer. This address is the one we will most often use. These three IP addresses are conveyed in Figure 5, for the case of vehicle *abe*.

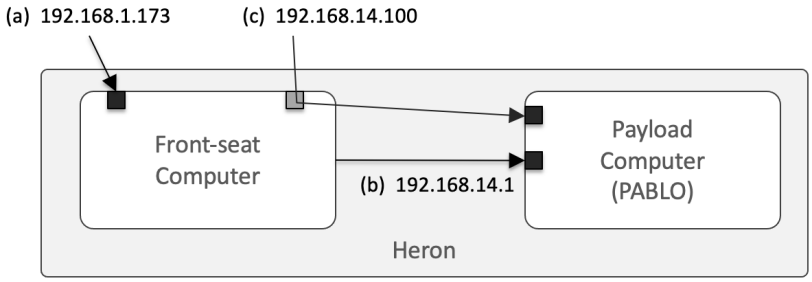


Figure 5: Each Heron has three IP addresses. (a) direct login to the front-seat, (b) login to the payload from the front-seat, (c) direct login to the payload computer. The example here shows the IP address for the vehicle *abe*.

You will not be able to log into the PABLO computer unless the front-seat computer is performing normally. If you cannot log into the PABLO, a first step in debugging is to see if you can log into the front-seat.

The IP addresses for the current pavlab Herons are in the table below.

4.3 IP Address Aliases

Since typing IP addresses requires a bit of attention at the keyboard and can lead to errors, GNU/Linux and OS X have a method for mapping IP addresses to more convenient names. The file `/etc/hosts` is a list of known static IP addresses referenced to aliases (type `man hosts` at the terminal prompt for more detail). If the file does not exist, create it and add the following, or if it does exist, append the following:

Vehicle	Front Seat IP	Front Seat IP to PABLO	IP assigned to PABLO
abe	192.168.1.173	192.168.14.1	192.168.14.100
ben	192.168.1.174	192.168.15.1	192.168.15.100
cal	192.168.1.169	192.168.16.1	192.168.16.100
deb	192.168.1.168	192.168.17.1	192.168.17.100
eve	192.168.1.167	192.168.18.1	192.168.18.100
fin	192.168.1.149	192.168.19.1	192.168.19.100
max	192.168.1.148	192.168.19.1	192.168.20.100
ned	192.168.1.147	192.168.21.1	192.168.21.100
oak	192.168.1.146	192.168.22.1	192.168.22.100
pip	192.168.1.145	192.168.23.1	192.168.23.100

Table 2: IP addresses for PABLO units and vehicles

```
192.168.14.100  abe
192.168.15.100  ben
192.168.16.100  cal
192.168.17.100  deb
192.168.18.100  eve
192.168.19.100  fin
192.168.20.100  max
192.168.21.100  ned
192.168.22.100  oak
192.168.23.100  pip
```

With the `/etc/hosts` file updated, mission files and command line references can use the IP alias name instead of the IP address. For example, you should be able to `ping` the robots now with:

```
$ ping deb
```

4.4 Bash Aliases for SSH and ping

You may also create aliases in your own shell environment, e.g., in your own `.bashrc` file on your laptop. See the list below. For example, the alias `sshaf` is short for "ssh to the abe front-seat computer", and `ssha` is short for "ssh to the abe pablo".

```
# Front-Seat Computer aliases (abe,ben,cal,deb,eve,fin,max,ned,oak)
alias sshaf='ssh -l student2680 192.168.1.173'
alias sshbf='ssh -l student2680 192.168.1.174'
alias sshcf='ssh -l student2680 192.168.1.169'
alias sshdf='ssh -l student2680 192.168.1.168'
alias sshef='ssh -l student2680 192.168.1.167'
alias sshff='ssh -l student2680 192.168.1.149'
alias sshmf='ssh -l student2680 192.168.1.148'
alias sshnf='ssh -l student2680 192.168.1.147'
alias sshof='ssh -l student2680 192.168.1.146'
alias sshpf='ssh -l student2680 192.168.1.145'

# Pablo aliases  (abe,ben,cal,deb,eve,fin,max,ned,oak)
alias ssha='ssh -l pabuser 192.168.14.100'
alias sshb='ssh -l pabuser 192.168.15.100'
alias sshc='ssh -l pabuser 192.168.16.100'
alias sshd='ssh -l pabuser 192.168.17.100'
alias sshe='ssh -l pabuser 192.168.18.100'
alias sshf='ssh -l pabuser 192.168.19.100'
alias sshm='ssh -l pabuser 192.168.20.100'
alias sshn='ssh -l pabuser 192.168.21.100'
alias ssho='ssh -l pabuser 192.168.22.100'
alias sshp='ssh -l pabuser 192.168.23.100'
```

Additionally, it is also often useful to have aliases for pinging the various computers. Similar to above, the alias `pingaf` is short for "ping the abe front-seat computer, and `pinga` is short for "ping the abe pablo".

```
# Front-Seat Computer aliases  (abe,ben,cal,deb,eve,fin,max,ned,oak)
alias pingaf='ping 192.168.1.173'
alias pingbf='ping 192.168.1.174'
alias pingcf='ping 192.168.1.169'
alias pingdf='ping 192.168.1.168'
alias pingef='ping 192.168.1.167'
alias pingff='ping 192.168.1.149'
alias pingmf='ping 192.168.1.148'
alias pingnf='ping 192.168.1.147'
alias pingof='ping 192.168.1.146'
alias pingpf='ping 192.168.1.145'

# Pablo aliases  (abe,ben,cal,deb,eve,fin,max,ned,oak)
alias pinga='ping 192.168.14.100'
alias pingb='ping 192.168.15.100'
alias pingc='ping 192.168.16.100'
alias pingd='ping 192.168.17.100'
alias pinge='ping 192.168.18.100'
alias pingf='ping 192.168.19.100'
alias pingm='ping 192.168.20.100'
alias pingn='ping 192.168.21.100'
alias pingo='ping 192.168.22.100'
alias pingp='ping 192.168.23.100'
```

For convenience, you can download a file with all these aliases, which should make copying into

your `.bashrc` file easier:

```
$ wget http://oceanai.mit.edu/pavlab/docs/heron_aliases
```

4.5 Assignment 2 (check off): Connecting to the network

This assignment validates that your laptop connects to the lab's WiFi. Be aware that to complete the labs you should not connect with the `MIT`, `MIT GUEST`, or `MIT SECURE` networks at the Pavilion. Connect using `kayak-local-5ghz`.

If not done already by preceding students, (a) move the antenna out from the lab and onto the dock (Figure 4), and (b) Plug in the power connector by the telephone inside the lab.

For the assignment, follow this checklist:

On your laptop...
[ ] Connect with the `kayak-local_5ghz` network
[ ] Enter the supplied router password when promted
[ ] Fill in your computer's IP address: `192.168.1._____`
[ ] Ping the front-seat IP address of the vehicle
[ ] Show the lab instructor the ping results

5 Using the PABLO Unit in a Vehicle

The PABLO unit used in the PABLO introductions lab will be installed in the vehicle and used to drive the `alpha_heron` mission. The system is designed so that the PABLO unit can move between simulation and the actual M300 vehicle with minimal changes.

5.1 Assignment 3 (self check-off): Connecting with a PABLO Unit to an M300

Follow these steps to connect with the PABLO:

[ ] Tighten the gland at the end of the PABLO unit's Ethernet plug
[ ] Make sure the two USB-B ports on your PABLO are capped
[ ] Get a battery box from the lab instructor
[ ] Place the PABLO unit in the payload bay (DON'T CONNECT POWER YET)
[ ] Connect the Ethernet plug to the M300's payload bay port
[ ] Connect power and wait approximately one minute
[ ] Use your laptop to `ssh` to the PABLO unit

Note, the PABLO will no longer have the IP address it had when it was directly connected to your laptop as in the previous lab. It should have the IP address corresponding to the name of your Heron. See the table in Section 4.2. For example, if your vehicle is *abe*, the PABLO IP address should be `192.168.14.100`.

5.2 Configuring the alpha_heron Mission

The `moos-ivp-heron` repository should already be present on your PABLO computer in the home directory. If it is not there, you can check it out using

```
PABLO $ cd
PABLO $ git clone git@github.com:pavlab-mit/moos-ivp-heron.git
```

This repository should have a `missions` folder with the particular mission `alpha_heron`. The goal in this step is to use this mission to deploy a Heron on the water.

We regularly tweak this baseline mission, so it is a good idea to update it on your PABLO before proceeding.

```
PABLO $ cd moos-ivp-heron
PABLO $ git pull
```

Note: as discussed in the previous PABLO setup lab, we use the convention of

```
PABLO $ command
```

for commands to be executed on your PABLO, and the solitary `$` prompt for commands to be executed locally on your laptop.

Configuring the Shoreside Launch Script

The shoreside community runs on your laptop. You should be able to simply launch the shoreside by invoking:

```
$ cd moos-ivp-heron/missions/alpha_heron
$ ./launch_shoreside.sh
```

Of course you will not be launching the shoreside with any time warp while you're working with vehicles on the water. After the shoreside is launched, make note of the IP address displayed at the top of the pMarineViewer window. You will need to know this IP address when configuring the mission launch script on your robot (so the robot knows where to find the shoreside computer).

Note: When or if you launch this shoreside on your laptop at home or elsewhere on campus on your home or MIT Wi-Fi network, you will get an IP address from that router. The one that counts is the IP address that you get when connecting the pavilion router (`kayak-local-5GHz`). This should begin with `192.168.1.X`. For convenience, rather than opening your Systems Network settings, your IP address should be shown a the top of your pMarineViewer window.

IMPORTANT NOTE: If your IP address shows up in pMarineViewer simply as "localhost", you will need to provide the IP address explicitly upon launch. So, determine your laptop's IP address, and launch instead with:

```
$ ./launch_shoreside.sh --ip=192.168.1.233  (or similar)
```

Should I make a copy of the alpha_heron mission?

It should be possible to run the `alpha_heron` mission today as-is, without any modification. Recall that it resides in `moos-ivp-heron/missions`. This is a read-only folder on your PABLO. So any changes you make are in jeopardy of not persisting.

If you would like to modify the mission in some form or another, just make a copy of the mission into your `moos-ivp-extend/missions` tree and folder. Presumably you have given yourself permission to edit and commit changes back to github.

Command Line Args When Launching your Vehicle

When you launch the vehicle MOOS community on your PABLO, on the Heron, you need to provide a few pieces of information:

- The name of the vehicle you are launching
- The IP address of the shoreside computer
- The location of the Waypoint pattern to be traversed.

The `launch_vehicle.sh` script contains command-line switches for all of the above, so we don't have to edit any `.moos` or `.bhv` files. You can run `./launch_vehicle.sh -h` at any time to see your options, but here we discuss the important ones. Here is an example of all three arguments:

```
PABLO $ ./launch_vehicle.sh --shore=192.168.1.224 --east
```

The shoreside IP address: Recall from above that you get this from your own laptop when it connects to the pavilion Wi-Fi. To specify it on the command-line: `--shore=192.168.1.224`, or whatever your IP address may be.

The survey region: This mission is configured to run a home plate pattern in one of two locations. In this mission we have not enabled collision avoidance with other Herons operating nearby. To mitigate this we configured East and West regions that do not overlap. Choose one, with either `--east` or `--west`. And try to pick the one the other group is not using at the moment.

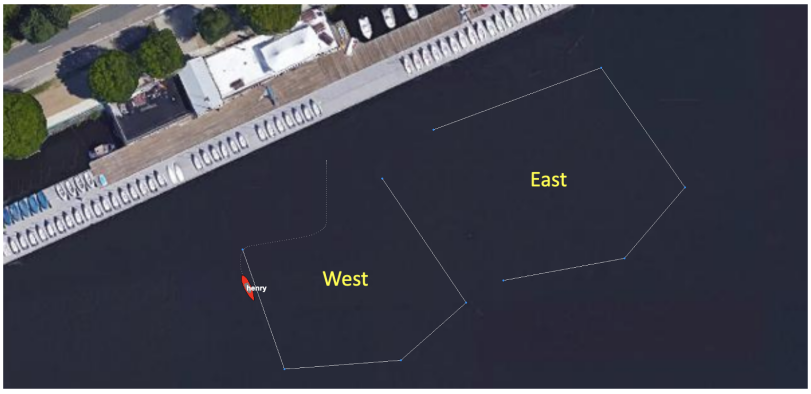


Figure 6: The East and West options for running the vehicle in the Alpha mission. Selectable in the `launch_vehicle.sh` script with either `--east`, or `--west`.

So, to launch your mission, here are some examples that you might encounter:

```
PABLO $ cd moos-ivp-heron/missions/alpha_heron
PABLO $ ./launch_vehicle.sh --shore=192.168.1.211 --east
PABLO $ ./launch_vehicle.sh --shore=192.168.1.214 --west
```

5.3 Assignment 4 (check-off): Launching the Mission on the Dock

This assignment verifies that your vehicle is ready to be controlled by the `alpha_heron` mission. **Do not let the motors run dry for more than a few seconds.**

In a terminal window on your laptop...
[ ] Launch the shoreside mission, making note of the IP address
In an ssh window to the PABLO unit...
[ ] `svn update` or `git update` anything that needs it
[ ] Prepare yourself to run the launch vehicle script: know your IP address, vehicle name, and east/west r
[ ] Launch the vehicle mission
In a terminal window on your laptop...
[ ] Re-launch the shoreside mission if necessary
[ ] When the vehicle displays on shoreside, deploy the mission
[ ] Stop the motors quickly
[ ] Show the lab instructor the operating mission

IMPORTANT NOTE: Make sure you do an `git pull` on the `moos-ivp-heron` folder. There are likely changes to the `alpha_heron` mission since your PABLO was configured.

5.4 Assignment 5 (check-off): Launching the Mission on the Water

This assignment verifies that your vehicle is ready to be controlled by the `alpha_heron` mission on the water. **Do not let the motors run dry for more than a few seconds.**

[ ] Ask the lab instructor to verify connections before deploying
[ ] Secure the payload bay lid
[ ] Re-check *all* connectors and caps on un-used ports
[ ] Push the vehicle back into the water
[ ] Remote-control the vehicle away from the dock
[ ] Deploy the mission for at least one loop of the pattern

5.5 Assignment 6 (check-off): Shutdown and Stowing

Shutting down and returning equipment is as important as any other portion of the lab. When working with the vehicles, always leave enough time to close out properly.

[ ] Return the vehicle to the dock

Retrieving...

[ ] Carefully remote control vehicle to dock (do not hit the dock)
[ ] Lift vehicle onto dock

Powering Down...

[ ] Through `ssh`, power down the PABLO unit
[ ] Open the payload bay and disconnect the PABLO unit
[ ] Turn off vehicle power using the main pushbutton switch

Returning Equipment...

[ ] Remove main computer battery from vehicle (the one with the handle)
[ ] Turn off remote control
[ ] Return remote to hook on the back wall of the lab
[ ] Place batteries on shelf to charge
[ ] Show lab instructor the batteries and vehicle

If you are the last operator...

[ ] Disconnect power to the repeater WiFi antenna
[ ] Bring in the repeater WiFi antenna inside and onto the storage hook

Completing the Lab

Your work on this lab is successful when you can demonstrate to the lab instructor the PABLO unit driving a vehicle on the river and after all equipment has been put away.